

Alex Daniel, Eric Fode
October 15 2025
CIS*3090
Dr. Denis Nikitenko

Assignment 1 Report

Introduction

The program we developed demonstrates the difference in efficiency between parallel and serial computation using Pthreads and a thread pool architecture. This efficiency is illustrated through the parallel computation of π , with results measured in terms of both accuracy and execution speed. Beyond this main objective, we also investigated how varying thread counts and task granularity affect performance. In completing the task, our goal was to implement a thread-safe task queue using mutexes and conditional variables, and to design a scalable thread pool architecture for efficient parallel computation of π .

Core Formulas and Algorithms

To implement this architecture, the program follows a producer-consumer pattern: the main thread reads the input file, enqueues tasks, and manages shutdown, while a fixed-size thread pool of worker threads processes tasks from the queue. The task queue is a thread-safe FIFO structure protected by a mutex and condition variables. Worker threads remain idle when no tasks are available and wake to compute π as tasks arrive. To perform the computation, we used the Leibniz series formula, iterating through a for-loop to approximate π . This formula is well-suited to our objective because it converges to π as the number of iterations increases, making improvements in computational efficiency directly observable and measurable.

Vital Data Structures

1. Task Structure

```
typedef struct Task {  
    long long iterations;  
    int taskId;  
    struct Task* next;  
} Task;
```

This structure encapsulates all the data required for a single π computation, allowing work to be easily passed between threads.

- The iterations field is a long long type because computing π accurately requires a very large number of iterations (e.g., 20 billion), preventing integer overflow with such large counts.
- The struct Task* next pointer enables the creation of a linked list for the task queue, which is necessary since the number of tasks is not known at compile time.

2. Thread-safe Queue

```
typedef struct TaskQueue {
    Task* head;
    Task* tail;
    size_t size;
    pthread_mutex_t mutex;
    pthread_cond_t cond;
} TaskQueue;
```

This structure enables multiple threads to safely add or remove tasks, following the producer-consumer pattern with condition variables. This design allows threads to avoid unnecessary waiting while maintaining thread safety.

- The `size_t` `size` field provides a quick way to check if the queue is empty without traversing it.
- The `pthread_mutex_t` `mutex` protects shared data from race conditions.
- The `pthread_cond_t` `cond` controls whether worker threads sleep or wake, preventing them from wasting CPU cycles constantly polling an empty queue.

3. Thread Pool Management

```
typedef struct ThreadPool {
    pthread_t* threads;
    int numThreads;
    // Shared state
    TaskQueue queue;
    volatile bool shutdown;
    long long tasksEnqueued;
    long long tasksCompleted;
    int availableWorkers;
    bool verboseOutput; // print res
    struct WorkerCtx* workerCtxArr;
} ThreadPool;
```

This structure serves as the central management hub for coordinating all worker threads, making the system easier to manage and debug.

- `pthread_t* threads` is an array representing the thread pool.
- `int numThreads` tracks the number of threads in the pool.
- `taskQueue queue` allows the thread pool to manage its work queue.
- The `volatile bool shutdown` flag stops all worker threads from completing tasks. It is declared `volatile` because in multithreaded code, one thread may change its value while another is reading it; without `volatile`, a compiler might cache the value or optimize repeated reads, potentially causing a thread to loop indefinitely.
- `tasksEnqueued` and `tasksCompleted` track the number of tasks, and when these counts are equal, shutdown is triggered.
- `bool verboseOutput` is set from a command-line argument, indicating whether the program should print the π computation summary to the terminal.
- Each thread can individually access these variables via the `struct WorkerCtx* workerCtxArr`.

4. Worker Context

```
typedef struct WorkerCtx {
    ThreadPool* pool;
    int workerIndex;
} WorkerCtx;
```

This structure provides each worker thread with the context it needs to operate.

- Without the `ThreadPool* pool` reference, a worker would have no way of knowing which pool it belongs to or its thread number. With this pointer, workers can access the thread pool's variables, such as `pool->queue` and `pool->shutdown`. Using a pointer also avoids copying the entire `ThreadPool` structure, ensuring efficient access.

New Parallel Terms Learned

1. `void* arg` as an argument

```
static void* workerMain(void* arg) {
    WorkerCtx* ctx = (WorkerCtx*)arg;
    ThreadPool* pool = ctx->pool;
    const int workerIndex = ctx->workerIndex;
```

One concept we were unfamiliar with before this course was passing `void* arg` as an argument to a thread function. This argument is essential because it allows thread-specific data to be passed into the function, giving each worker access to the shared thread pool and its own ID, effectively defining the worker.

2. Broadcast from the pthreads library

```
pool->tasksCompleted += 1;
pthread_cond_broadcast(&pool->queue.cond);
```

In the program, when a task is completed, we use the `pthread_cond_broadcast` function instead of `pthread_cond_signal` to wake all threads. This is done whenever a task finishes or the pool begins shutdown, as either event can change the condition for all threads. After being awakened, a worker thread can do one of three things:

- Call `queuePop()` to remove and process the next task.
- Go back to sleep until a new task is enqueued or shutdown begins.
- Exit by unlocking the mutex and terminating the thread loop.

3.1 Correctness Analysis

Set up and execution of the Test Script

You first need to set up executable rights for shell scripts, which can be done through the following command: ``chmod +x tests/run_tests.sh``

The test script can now be run with the following command: ``./tests/run_tests.sh``

The script ensures the `A1.c` file will be made using the `make` command, it will then describe what each scenario is testing for, and it then redirects the logs to a dedicated log file located at: ``tests/test_results.log``

Purpose of the Tests

1. To verify the program's correctness and robustness across these various inputs: normal, boundary, and invalid.
2. Ensure input handling, PI computational accuracy, and thread synchronization are all reliable.
3. Ensure no memory leaks or data race conditions exist across all test scenarios.

Testing Approach

Each Scenario executed a test for the following three cases:

1. Base Run: validates the expected output of pi
2. Valgrind [leak-check=full]: validates no memory leaks or invalid memory accesses
3. Valgrind [tool=drd]: validates there are no race conditions or deadlocks within the multithreaded execution.

The log outputs are directed to ``tests/test_results.log`` for manual verification.

Test Scenarios

Test ID	Scenario	Purpose	Expected Outcome
t00	Single Thread (1 thread, 1 task)	The baseline test allows for a comparison of parallel output to serial output.	Pi should be calculated, no memory leaks, no race conditions.
t01	An equal number of threads and tasks (8 threads, 8 tasks). Same iterations per task	Validates parallel consistency and speedup using even and consistent workloads.	Correct Pi, constant results, no memory leaks, no race conditions.
t02	An equal number of threads and tasks (8 threads, 8 tasks). Different iterations per task	Perform a stress test to validate performance under uneven workloads per thread.	All tasks complete properly, no memory leaks, no race conditions.
t03	More tasks than threads available (4 threads, 8 tasks). Same iterations per task	Validates if idle thread handling and scheduling are correct.	Threads should exit cleanly, there should be no hangs, no memory leaks, no race conditions.
t04	More tasks than	Validates the same as	Same as t04 with a stress

	threads available (4 threads, 8 tasks). Different iterations per task	t03 with uneven workloads to ensure queueing and scheduling work as intended.	test due to uneven workloads.
t05	Empty text file (0 values within file)	Ensures edge case where an empty file is listed is handled gracefully. Fails fast, and leave a message.	Quick fail with an error message, no memory leaks, no race conditions.
t06	Thread pools size set to 0 (0 threads, 1 task)	Similar to t05, ensure scenario fails fast and gracefully.	Similar to t05, ensure scenario fails fast and gracefully.
t07	The thread pool size is set to a negative value (-1 threads, 1 task)	Similar to t05, ensure scenario fails fast and gracefully.	Similar to t05, ensure scenario fails fast and gracefully.
t08	No tasks set (8 threads, 0 tasks)	Ensures the program can handle an empty workload gracefully.	Ensures threads start up and terminate instantly, no memory leaks, no race conditions. There should be no output
t09	Large iteration size of 10 billion (1 thread, 1 task)	Test performance, stress and overflow safety.	Pi is computed as normal, execution completes safely, no overflow, no memory leaks, no race conditions.
t10	More threads than tasks available (8 threads, 4 tasks)	Ensures the task queue is robust and properly shuts down when all tasks are complete.	The extra threads wait for worker threads to complete, then shut down cleanly, no memory leaks, no race conditions.
t11	A lot of tasks: 300 total, with some threads (8 threads, 300 tasks)	Test the scheduler's implementation through high stress loads, and test the throughput.	All tasks are completed properly, no memory leaks, no race conditions.
t12	Negative iterations set for a task (4 threads, 4 tasks) With 3 negative tasks	Input validation for each task	Should fail fast on a task and report an error/input validation error. No memory leaks, no race conditions.
t13	Malformed Input	Tests basic malformed	Does not crash, exits

	(random text / invalid data)	input error handling.	safely, has no memory leaks, and has no race conditions.
--	------------------------------	-----------------------	--

Scenarios are identified by t## due to the lexicographical ordering used by shell scripts when listing file names.

Output Verification

- Function / Computational Correctness: Compare the computed Pi value to a baseline value within a set tolerance range based on iterations per task.

```
----- Base Run -----
Thread 1 completed computed Pi using 5000000 iterations, the result is 3.141592453590
```

- Memory Correctness: All tests run with valgrind --leak-check=full should report that all heap blocks were freed – no leaks are possible & ERROR SUMMARY: 0 errors from 0 contexts.

```
----- Valgrind Run -----
==91660== Memcheck, a memory error detector
==91660== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==91660== Using Valgrind-3.19.0 and LibVEX; rerun with -h for copyright info
==91660== Command: ./A1 tests/inputs/t00_single_thread_small.txt true
==91660==
Thread 1 completed computed Pi using 5000000 iterations, the result is 3.141592453590
==91660==
==91660== HEAP SUMMARY:
==91660==    in use at exit: 0 bytes in 0 blocks
==91660==   total heap usage: 7 allocs, 7 frees, 1,816 bytes allocated
==91660==
==91660== All heap blocks were freed -- no leaks are possible
==91660==
==91660== For lists of detected and suppressed errors, rerun with: -s
==91660== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

- Thread Correctness: The DRD report should show no errors detected: ERROR SUMMARY: 0 errors from 0 contexts.

```
----- DRD Run -----
==91681== drd, a thread error detector
==91681== Copyright (C) 2006-2020, and GNU GPL'd, by Bart Van Assche.
==91681== Using Valgrind-3.19.0 and LibVEX; rerun with -h for copyright info
==91681== Command: ./A1 tests/inputs/t00_single_thread_small.txt true
==91681==
Thread 1 completed computed Pi using 5000000 iterations, the result is 3.141592453590
==91681==
==91681== For lists of detected and suppressed errors, rerun with: -s
==91681== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 16 from 8)
```

3.2 Performance Analysis

Setup for Experiments

- Windows 10 system
- 24 core system
- GCC with -O2 optimization
- Timing method follows “date +%s%3N” for millisecond precision
- 30 repetitions are done per configuration
- 20 billion iterations across all test configurations

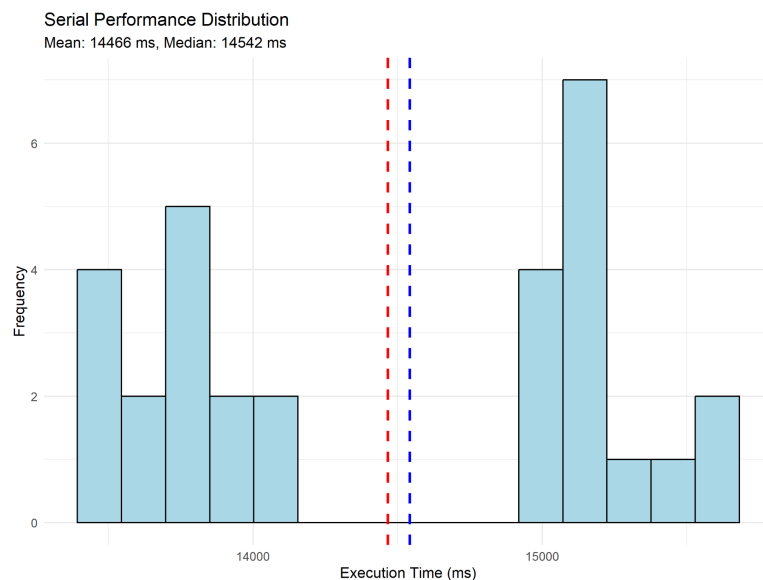
Serial Version Results

The serial program uses a single thread to execute all 20 billion iterations, serving as a baseline for measuring the speedup of the parallel program. Multiple runs were performed, as a single run would complete too quickly to measure accurately and to better capture variability in execution time. Across 30 runs of the 20 billion iterations, the results are as follows:

- Average time (mean): 14,466 ms
- Median time: 14,542 ms
- Standard Deviation: 678 ms (5.3% coefficient of variation)

Overall, the serial baseline demonstrates consistent performance, as reflected by the low standard deviation, and provides a theoretical minimum runtime for execution on a single core.

We can see the serial version results on the bar graph here:



The red line is mean and the blue line is median.

Thread Scaling Analysis (Parallel)

In our parallel experiments, two major dimensions were explored:

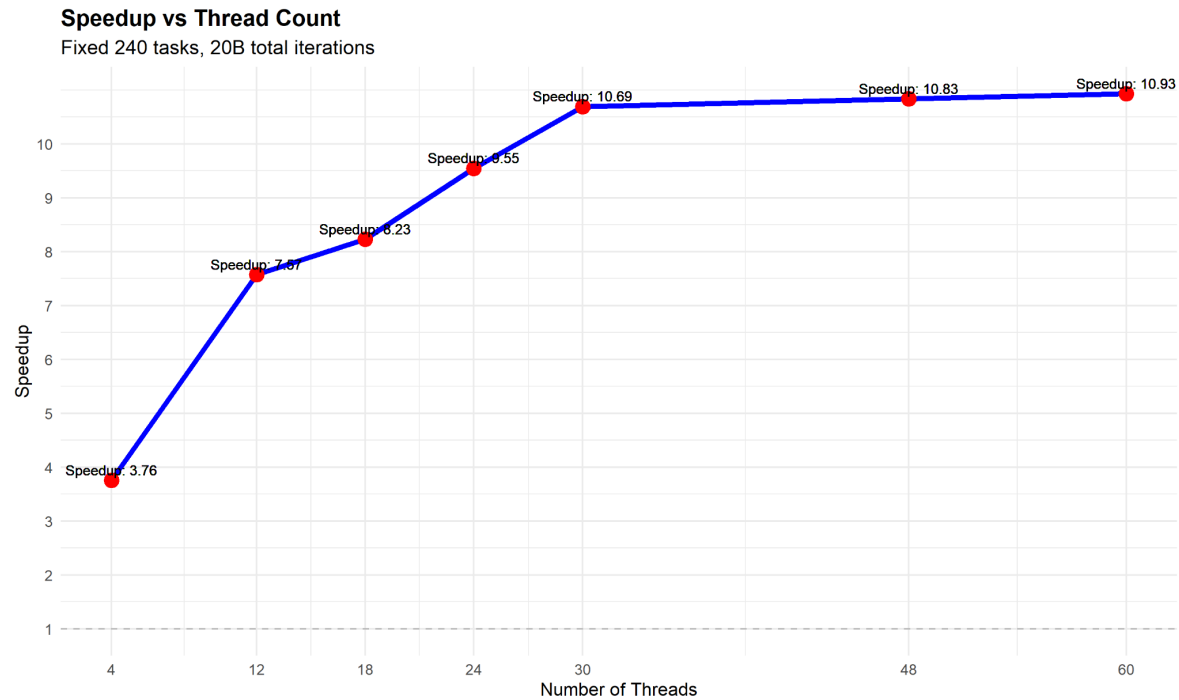
- Thread scaling: 4, 12, 18, 24, 30, and 48 threads (with 240 total tasks)
- Task scaling: 4, 12, 18, 24, 30, and 48 tasks (with 24 threads fixed)

The thread scaling tests measure how performance changes as we increase the number of threads while keeping total work and number of tasks the same.

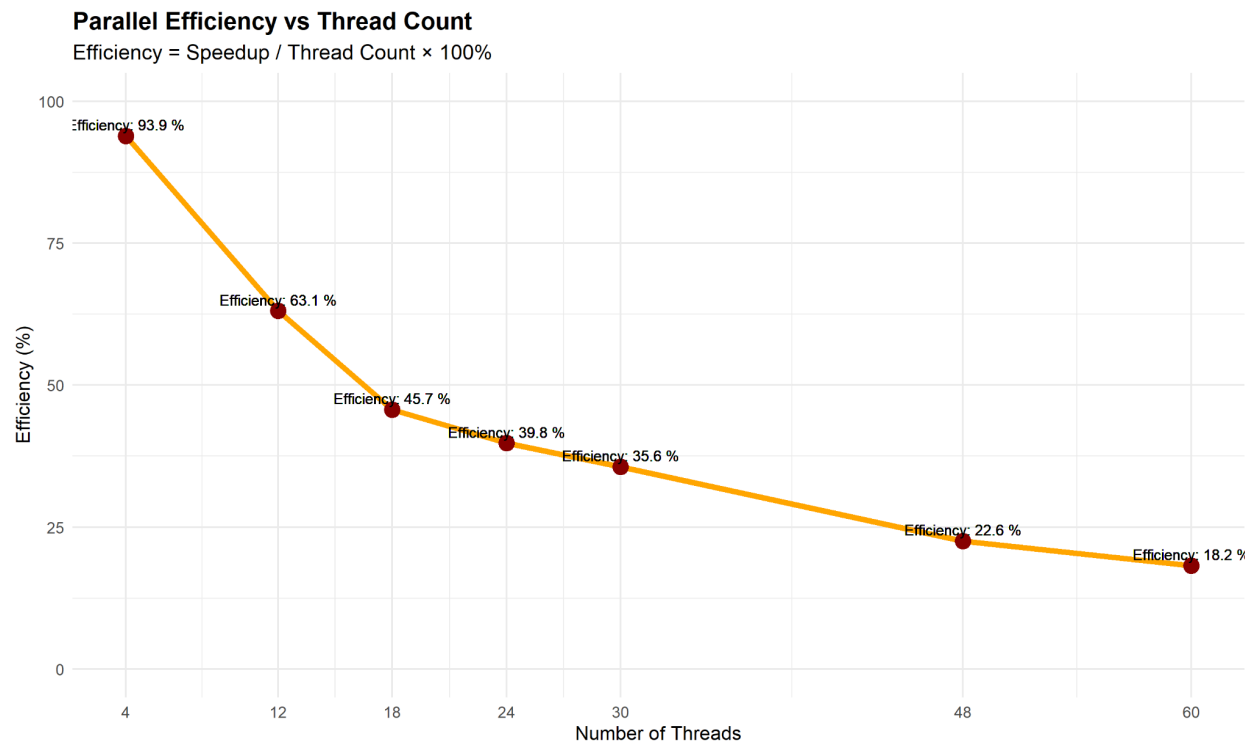
Threads	Mean Time (ms)	Speedup	Efficiency
4	3,851	3.76x	93.9%
12	1,911	7.57x	63.1%
18	1,758	8.23x	45.7%
24	1,515	9.55x	39.8%
30	1,353	10.69x	35.6%
48	1,335	10.83x	22.6%
60	1,323	10.93x	18.2%

Including the 4-thread configuration highlights the near-linear scaling observed at low thread counts. With 4 threads, the program achieves a 3.76× speedup and 93.9% efficiency, indicating minimal synchronization overhead and excellent CPU utilization. As the number of threads increases, performance initially improves but begins to plateau after 30 threads. This behavior is expected, as our system has 24 physical cores; close to where the speedup levels off. The difference in speedup between 30 and 48 threads is minimal because additional threads introduce context-switching and mutex contention, resulting in negligible performance gains. This plateau illustrates the limits of hardware parallelism: once the number of threads exceeds the number of available cores, additional threads contribute little to overall speedup, while efficiency continues to decline. Based on these results, the optimal number of threads for this system is around 30, roughly matching the number of physical cores.

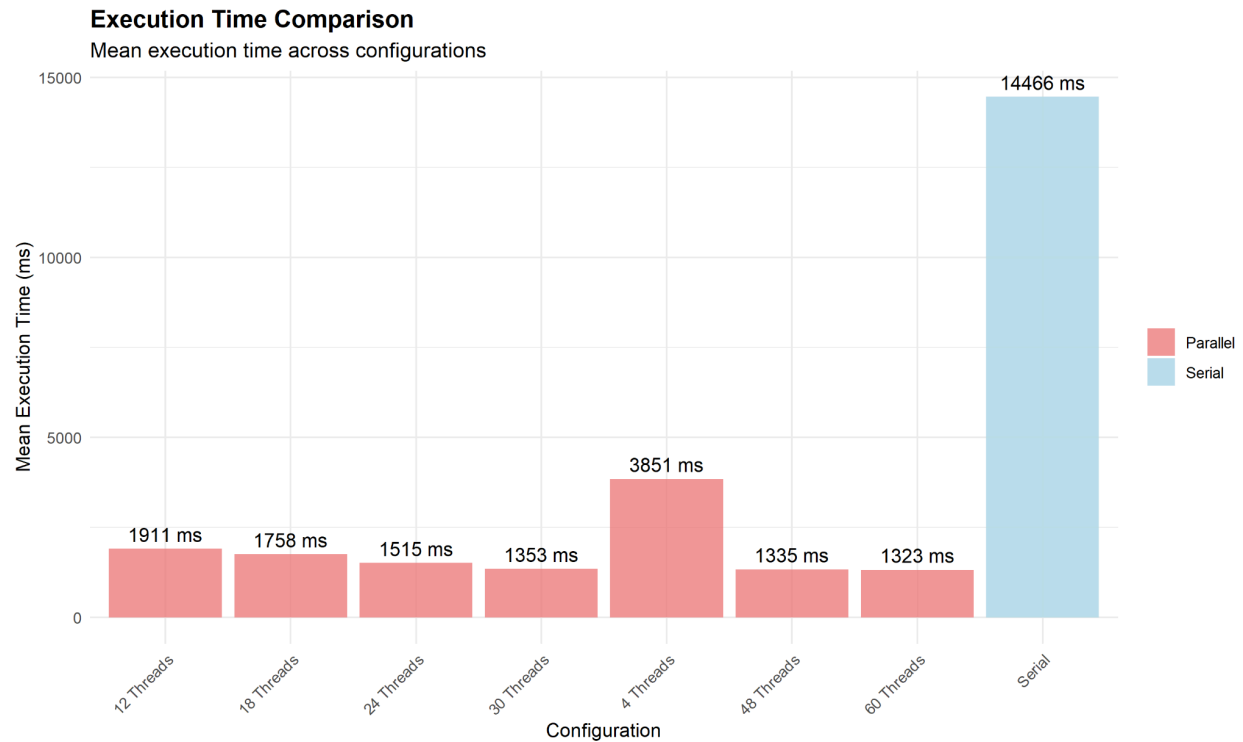
The following line graph shows that speedup increases significantly until the number of threads approaches the system's core count, after which the curves plateau.:



This graph shows efficiency steadily decreasing as the number of threads increases:



The accompanying bar graph clearly illustrates how much faster the parallel program is compared to the serial execution, even with just four threads:



Task Granularity Analysis (Parallel)

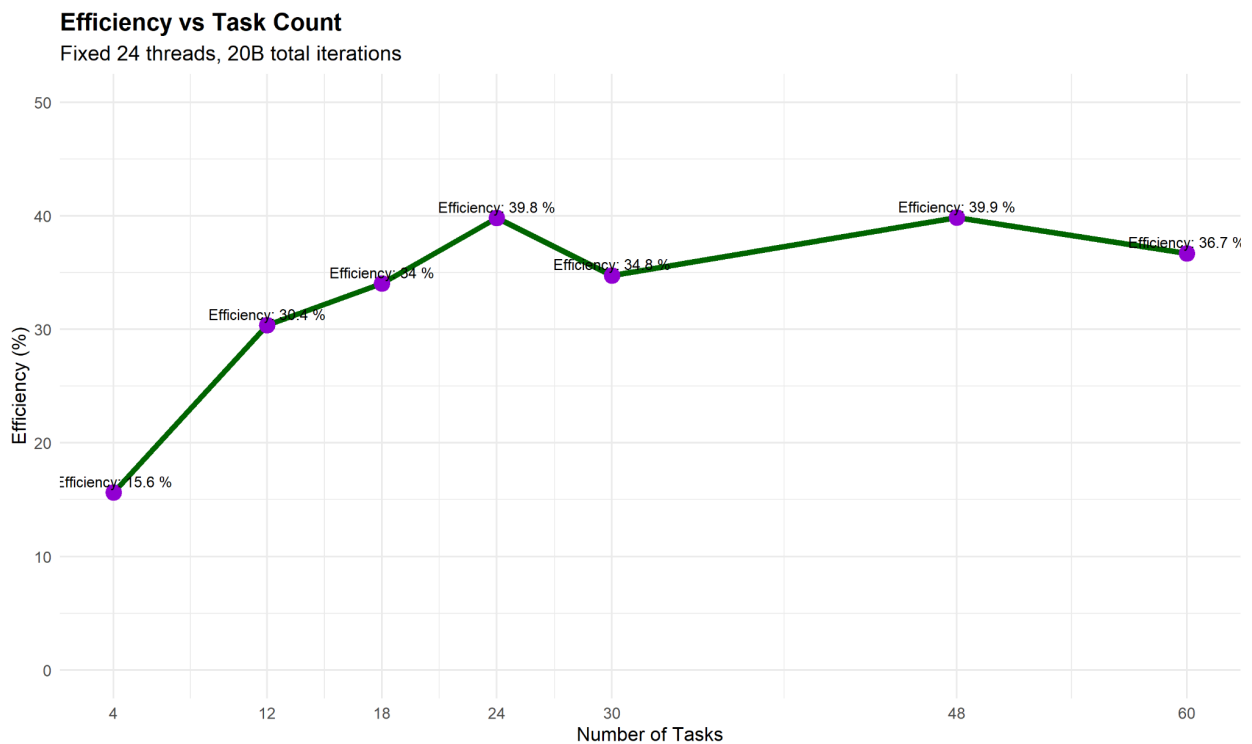
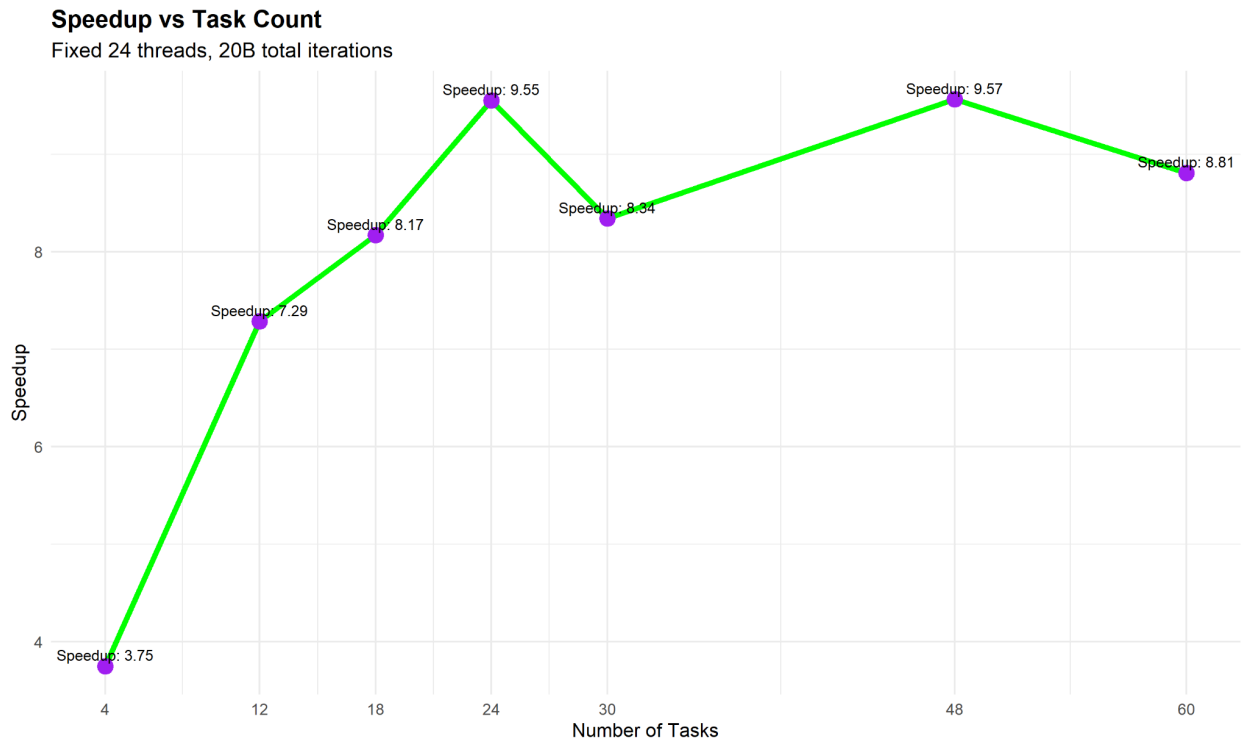
In the task scaling tests, we fixed the thread count at 24 and varied the number of tasks, each totaling 20 billion iterations. This allows us to examine how different workloads per task affect performance.

Tasks	Iterations per Task	Mean Time (ms)	Speedup	Efficiency
4	5,000,000,000	3856	3.75x	15.56%
12	1,666,666,667	1985	7.29x	30.4%
18	1,111,111,111	1770	8.17x	34%
24	833,333,334	1514	9.55x	39.8%
30	666,666,667	1734	8.34x	34.8%
48	416,666,667	1512	9.57x	39.9%
60	333,333,334	1642	8.81x	36.7%

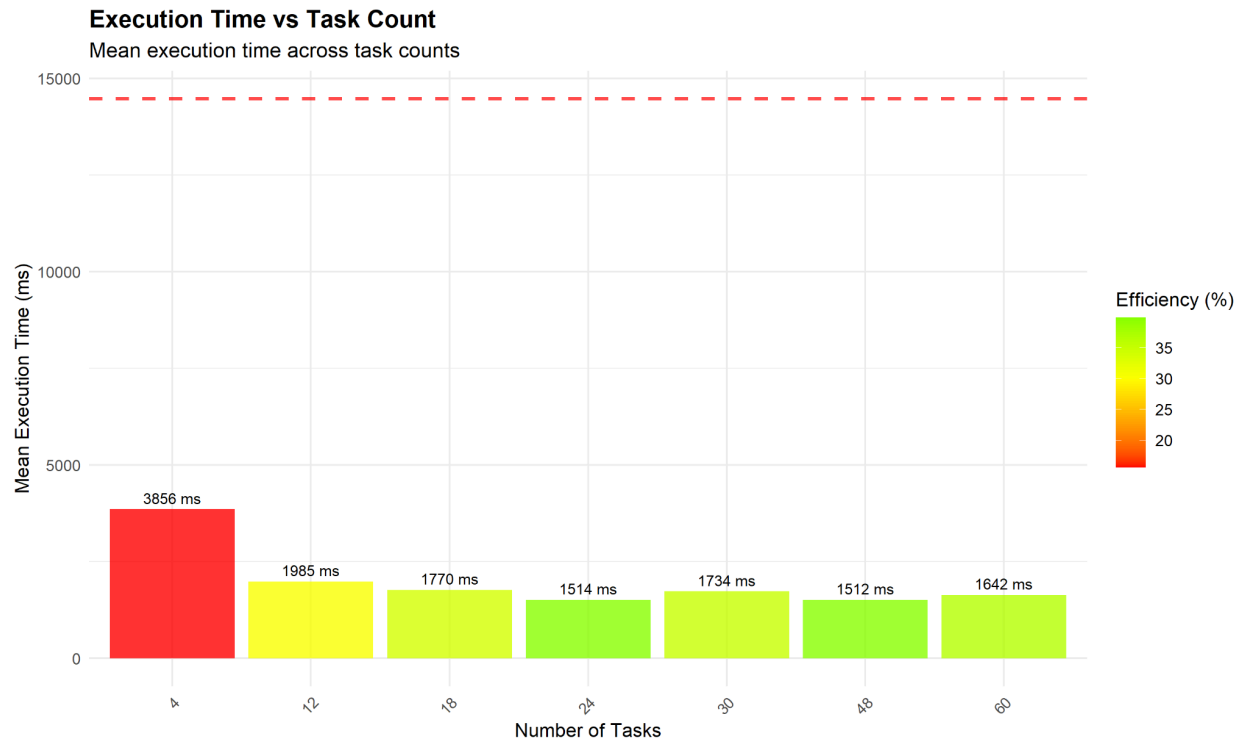
The 4-task case represents coarse granularity: each task runs for a long period, resulting in poor load balancing, as idle threads must wait for others to finish. As the number of tasks increases to match the thread count, load distribution improves significantly, minimizing idle time. However, when the number of tasks exceeds the thread count, tasks become too

fine-grained, increasing synchronization overhead from frequent queue operations and causing efficiency to plateau. The optimal balance appears to be around 24 tasks, where overhead and load distribution are best balanced.

The following line graphs shows a plateau in speedup and efficiency once the number of tasks exceeds the thread count:



The bar graph demonstrates that performance improves as we approach 24 tasks and then levels off beyond that point:



Comparative Analysis: Serial vs Parallel (Conclusion)

The best-performing configuration (30 threads, 240 tasks) completed the 20-billion-iteration workload in 1,353 ms, compared to 14,466 ms for the serial version; a 10.69 $\times$ speedup with an efficiency of 35.6%. This efficiency reflects the practical limits of our 24-core system, where hardware saturation and synchronization overhead begin to outweigh the benefits of additional threads. Our thread-scaling experiments intentionally tested configurations both below and above the core count (e.g., 12, 18, 24, 30, and 48 threads) to observe the effects of over-subscription. The performance plots clearly showed diminishing returns beyond 24 threads, with slight slowdowns occurring as thread management overhead increased and cores began sharing execution time.

According to Amdahl's Law, if approximately 7.5% of the program remains serial and 92.5% is parallelizable, the theoretical maximum speedup on infinite cores is about 13.3x, while the predicted limit on 30 threads is about 9.9x. Our measured speedup of 10.69x slightly exceeds that estimate, suggesting the serial portion is even smaller than 7.5% and that our implementation makes highly efficient use of available parallelism. Overall, the results confirm that performance scales well up to the number of physical cores, then plateaus as synchronization and scheduling costs dominate. Increasing threads beyond the hardware limit does not improve runtime and may even degrade it. These findings demonstrate that optimal performance is achieved when the number of worker threads roughly matches the number of CPU cores, ensuring balanced utilization without oversubscription.

Here in the graph the blue line represents the theoretical speedup of Amdahl's law and the red dots are what we got in the observed data:

Amdahl's Law vs Observed Performance

Serial fraction: 7.5 %, Parallel fraction: 92.5 % - Observed exceeds prediction

